Problem Set 2: Manifold Interpolation and Graph Neural Networks

Geometric and Topological Methods in Machine Learning
CPSC 6440 01 / AMTH 3220 01 / CPSC 4844 01 / MATH 5220 01 / MATH 7440 01

Assigned: October 2
Due: October 25, 11:59pm
2025

1 Graph Neural Networks for Single-Cell Analysis

Graph convolutional networks (GCNs) have emerged as powerful tools for analyzing biological networks, particularly in single-cell genomics where cells can be naturally represented as nodes in a graph connected by similarity relationships. For reference, see Kipf and Welling. You may also find this review helpful: "How Powerful are Graph Convolutional Networks?". We have provided you with the skeleton code. **Your task is to fill in the functions of the code answering the below questions in the following notebook**: Notebook Link. **Note that 1.1, 1.2 and 1.3 are all in the same notebook link.** Please first save a copy on your drive by going to File → Save a copy on your Drive and then fill in the notebook in your saved version. We have also attached a copy of this notebook on Canvas if you wish to work on it on your local computer.

1.1 Constructing k-NN Graph and Building a GCN for Single-Cell Data

Load the PBMC12k dataset from 10X Genomics using `sc.read_10x_h5()` or `sc.read_10x_mtx()`. After preprocessing (filtering, normalization, selecting highly variable genes), construct a k-nearest neighbor graph using the PCA-reduced feature space.

Your task is to fill out the skeleton code for `BetterGCNConv` in `GNN-HW.ipynb` to build an improved GCN, following Xu et al. Reference the PyTorch Geometric tutorial on Creating Message Passing Networks, and implement these modifications:

- Use a multi-layer perceptron (MLP) for the message transformation function γ.
- Do not perform normalization in the aggregation step ϕ.

Question 1.1. *Experiment with different values of k for graph construction and report how connectivity affects downstream performance.*

1.2 Performing Cell Type Classification

Your task is to perform cell type classification by following the workflow in the same notebook provided (Notebook Link) as given below:

1. **Leiden Clustering:** Cluster your dataset using Leiden Clustering (Please see HW1 for how to do that).
2. **Use Leiden clusters as labels:** In biology, experts often gate cells into types using a few marker genes. Here, we instead use the Leiden clusters from the previous step as cell type classification labels to generalize without expert annotations.
3. **Train a GCN to classify cells:** Now train your GCN to classify withheld cells according to these labels. Report your test accuracy and generate a confusion matrix.

1.3 Gene Expression Denoising using MAGIC and GCN

- Here we will look at gene expression denoising from a graph signal processing perspective. Single-cell RNA sequencing data are typically represented as a cell $\times$ gene matrix, where each row corresponds to a cell and each column to a gene. Instead, we represent genes as real-valued signals over a cell-cell graph where each cell can be represented as a node. The noise in scRNA-seq measurements are irregular, high-frequency fluctuations in these graphs signals. Hence, **denoising genes** is fundamentally a signal-level task, where we our goal is to recover smooth biologically meaningful gene expression patterns by filtering these real-valued signals defined on the cell-cell graph.

- In order to achieve this, use the k-nearest neighbors (kNN) graph you constructed in 1.1 where edges connect cells with similar expression profiles. This graph captures the intrinsic structure of the cell population and will serve as an input to your GCN model.

- **Your task is to train the GCN to predict denoised gene expression values (Notebook Link)**. The model should learn a mapping from noisy gene expression profiles (input node features) to smoother signals (outputs), leveraging message-passing across the cell-cell graph. This addresses a key issue in single-cell RNA sequencing where roughly 95% of the genes are dropped out - meaning that many transcripts are not detected or measured during sequencing.

- However, this task is challenging because we do not have access to the ground truth denoised gene expression values, making it difficult to train a denoising GCN model.

- To overcome this, **your task (Notebook Link) will be to make use of an analytical graph signal processing method called MAGIC** (Markov Affinity-based Graph Imputation of Cells) van Dijk et al. to compute imputed values for some genes, which are then use for training the GCN - allowing it to generalize to remaining genes.

- MAGIC works by computing the data diffusion operator P (As done in HW1) from your k-NN graph, and diffuse each gene vector g for t steps:
$$\hat{g} = P^t g \tag{1}$$

- This procedure denoises gene values by suppressing high-frequency noise on the cell-cell graph while preserving the manifold structure. Use the denoised values $\hat{g}$ as node-level regression labels for your GCN.

Question 1.2. *Visualize predicted vs. true expression for several genes by experimenting with different t values.*

Question 1.3. *How does graph connectivity (different k values) influence denoising quality?*

2 Embryoid Body Dataset

The Embryoid Body (EB) Dataset Moon et al. contains single-cell RNA-seq profiles collected over a 27-day differentiation EB differentiation time course, plus bulk RNA-seq from FACS-sorted EBs surface markers.

3 Implementing the PHATE Regularized Autoencoder

Here we will learn how to implement the first stage of the pipeline, a PHATE Moon et al. regularized autoencoder. The encoder takes in high-dimensional single cell data and compresses it into a lower-dimensional latent representation. The decoder then reconstructs the original data from the latent representations, ensuring that the latent space preserves as much biological signal as possible. In order to make the latent space meaningful, we introduce a PHATE regularization head. The latent representation is linearly mapped into PHATE coordinates, and the model is penalized if the prediction deviates from the actual PHATE embeddings of the data.

We have provided you with the skeleton code. **Your task is to fill in the functions of the code in the following notebook**: Notebook Link. Please first save a copy on your drive by going to File → Save a copy on your Drive and then fill in the notebook in your saved version. We have also attached a copy of this notebook on Canvas if you wish to work on it on your local computer.

Question 3.1. *Given that cellular data often contains noise, how would you adapt the model into a denoising autoencoder to better recover the underlying signal?*

4 Implementing a Neural ODE to generate trajectories

Once we have a biologically meaningful latent space, we model how cells move through the space over time using an ordinary differential equation (ODE) network. Let $x(t) \in \mathbb{R}^d$ denote a point in latent space at time t, and let $f_\theta : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ be a neural network parameterized by θ. We define the dynamics as:

$$\frac{d}{dt}x(t) \;=\; f_\theta\big(t, x(t)\big), \qquad x(t_i) = x_i \in X_{t_i}. \tag{2}$$

Given $X_{t_i} \in \mathbb{R}^{n_i \times d}$, where n_i is the number of samples, and d is the latent dimension sampled at times $t_0 < \cdots < t_T$, we integrate the ODE from X_{t_i} to predict $\hat{X}_{t_{i+1}}$ at the next time. We train f_θ by minimizing the sum of three losses: an optimal transport loss, an energy regularizer, and a density loss.

We have provided you with the skeleton code in the same note book as above: Notebook Link. **Your task is to implement the following three loss functions:** Please first save a copy in your drive by going to File → Save a copy in Drive and then fill in the notebook in your saved version. We have also attached a copy of this notebook on Canvas if you wish to work on it on your local computer.

1. **Optimal Transport Loss:** In this loss, we match the predicted data $\hat{X}$ to the ground truth Y using the 2-Wasserstein distance with Euclidean ground cost as:

$$\mathcal{L}_{\mathrm{OT}}(\hat{X}, Y) \;=\; W_2^2(\hat{\mu}, \nu) \;=\; \inf_{\pi \in \Pi(\hat{\mu}, \nu)} \int \|x - y\|_2^2 \, d\pi(x, y), \tag{3}$$

   Hint: You may use the Python OT library to compute μ, ν and the 2-Wasserstein distance (emd2 module) and PyTorch to compute the Euclidean ground cost.

2. **Energy Loss:** In order to encourage smooth flows, we penalize the vector-field magnitude along the learned trajectories. For time grid $\{t_k\}_{k=1}^T$ and trajectories $\{x_i(t_k)\}$ obtained by integrating from initial points, we define the energy loss as:

$$\mathcal{L}_e \;=\; \frac{1}{nT} \sum_{k=1}^{T} \sum_{i=1}^{n} \big\| f_\theta(t_k,\, x_i(t_k)) \big\|_2^2. \tag{4}$$

   Hint: Use the odeint function to compute the full trajectory and then evaluate the vector field along the trajectory.

3. **Density Loss:** To keep predictions within or near the data manifold at each time, we use a density regularized loss defined as hinge loss on k-nearest neighbor distances:

$$\ell_{\mathrm{density}}(x, t) = \frac{1}{k} \sum_{j=1}^{k} \max\big(0,\, d_{(j)}(x(t)) - h\big), \qquad \mathcal{L}_d(\hat{X}, Y) = \frac{1}{|\hat{X}|} \sum_{x \in \hat{X}} \ell_{\mathrm{density}}(x, t), \tag{5}$$

   where $d_{(j)}(x(t))$ is the distance to the j-th nearest neighbor of $x(t)$ in Y, k is a small integer, and $h > 0$ is a hinge margin.

   Question 4.1. *Experiment with the Neural ODE density loss by sweeping the k and hinge loss hyperparameters and briefly report the effects.*

Question 4.2. *What is unbalanced optimal transport, and how would you implement it in this system? Please provide a pseudocode or explanation.*

Question 4.3. *How does the network learn manifold geodesics? Describe the losses and how they promote the computation of manifold geodesics.*

5 Submission Instructions

Each student should submit the following files to **Gradescope** by **October 25, 11:59pm** :

1. A detailed write-up in pdf form, titled `[last name]_[first name]_ps2_report.pdf`
2. A pdf of both the notebooks with the code and all the outputs including visualizations, titled `[last name]_[first name]_ps2_notebook_1.pdf` and `[last name]_[first name]_ps2_notebook_2.pdf`
3. A .ipynb file of the filled in notebooks, titled `[last name]_[first name]_ps2_notebook_1.ipynb` and `[last name]_[first name]_ps2_notebook_2.ipynb`

Works Cited

Kipf, Thomas N., and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. 2017. *arXiv*, arxiv.org/abs/1609.02907.

Moon, Kevin R., et al. "Visualizing structure and transitions in high-dimensional biological data". *Nature Biotechnology*, vol. 37, no. 12, Dec. 2019, pp. 1482–92. https://doi.org/10.1038/s41587-019-0336-3.

Van Dijk, David, et al. "Recovering Gene Interactions from Single-Cell Data Using Data Diffusion". *Cell*, vol. 174, no. 3, July 2018, 716–729.e27. https://doi.org/10.1016/j.cell.2018.05.061.

Xu, Keyulu, et al. How Powerful are Graph Neural Networks? 2019. *arXiv*, arxiv.org/abs/1810.00826.